

ai修改设计报告ver1.4

DXF → SAM 几何转换工具 — 设计报告

版本：v1.0

日期：2026-07-25

项目：哈尔滨工程大学 国卓班第二期 SAM 二次开发 — 任务7：DXF 图纸导入 SAM

1. 项目概述

1.1 建设背景

船舶设计与制造过程中，大量图纸以 AutoCAD DXF 格式存在。这些图纸需要在 SAM（Ship Analysis Model）平台中进行三维建模和 CAE 分析。当前缺少一条从 DXF 图纸到 SAM 模型的自动化转换通道，依赖人工重新建模，效率低且易出错。

本项目旨在开发一套 DXF → SAM 的自动转换工具，以插件形式运行于 SAM 软件中，解决船舶图纸中直线、圆弧、样条曲线等几何的解析与重建问题。

1.2 建设目标

目标	说明
核心目标	将 DXF 图纸中的几何图元自动转换为 SAM 三维模型
质量目标	曲线拟合偏差 $\leq 0.1\text{mm}$ （可配置）；不因任何 DXF 文件崩溃
效率目标	100MB 文件解析时间 ≤ 30 秒；支持异步处理
易用目标	提供图形化界面，支持选择导入基点

1.3 用户场景

用户	场景	需求
CAE 工程师	将 CAD 图纸导入 SAM 进行结构分析	完整还原图纸几何，支持基点定位

项目验收方	批量验证多张图纸的转换效果	统计数据清晰，异常可追溯
-------	---------------	--------------

1.4 范围与约束

处理范围：

图元类型	处理方式	优先级
直线 (LINE)	直接映射为 SAM 直线	P0
圆 (CIRCLE)	直接调用 SAM 画圆接口	P0
圆弧 (ARC)	离散化为多段线后逐段绘制	P0
多段线 (LWPOLYLINE)	解析顶点序列后逐段绘制	P0
样条曲线 (SPLINE)	调用 OCC 库离散化为多段线	P1
椭圆/椭圆弧 (ELLIPSE)	调用 OCC 库离散化为多段线	P1
倒角 (CHAMFER)	解析并转换为 SAM 格式	P1

不包含范围：

- 不包含对块（BLOCK）与外部依赖的解析，若图纸中有块与外部依赖，需要用户手动将其炸开后再导入
- 不对螺旋线、圆环与修订曲线进行解析
- 不包含对图层进行解析，不包含尺寸标注
- 不支持二进制格式的 DXF 文件

技术约束：

- 必须通过 SAM 的 skcGeomFactory 接口（CreatePoint / CreateLine / CreateCircle）绘制
- 使用 libdxfrw 作为 DXF 解析库
- 曲线离散化基于 OpenCASCADE（OCC）数学库
- C++ 开发，CMake 构建，Qt 做 GUI
- 以插件形式在 SAM 软件中运行
- 假设用户使用世界坐标系，且单位为毫米

1.5 术语定义

术语	说明

DXF	AutoCAD 的开放图纸交换格式
SAM	Ship Analysis Model，三维建模/CAE 平台
图元	DXF 中的基本几何对象（LINE, ARC, CIRCLE 等）
离散化	将连续曲线按容差转换为多段线的过程
OCC	OpenCASCADE，开源几何建模内核，提供曲线离散化算法
基点	用户指定的 CAD 原点在 SAM 场景中的放置位置
MVP	最小可行性产品，第一迭代目标

2. 总体架构

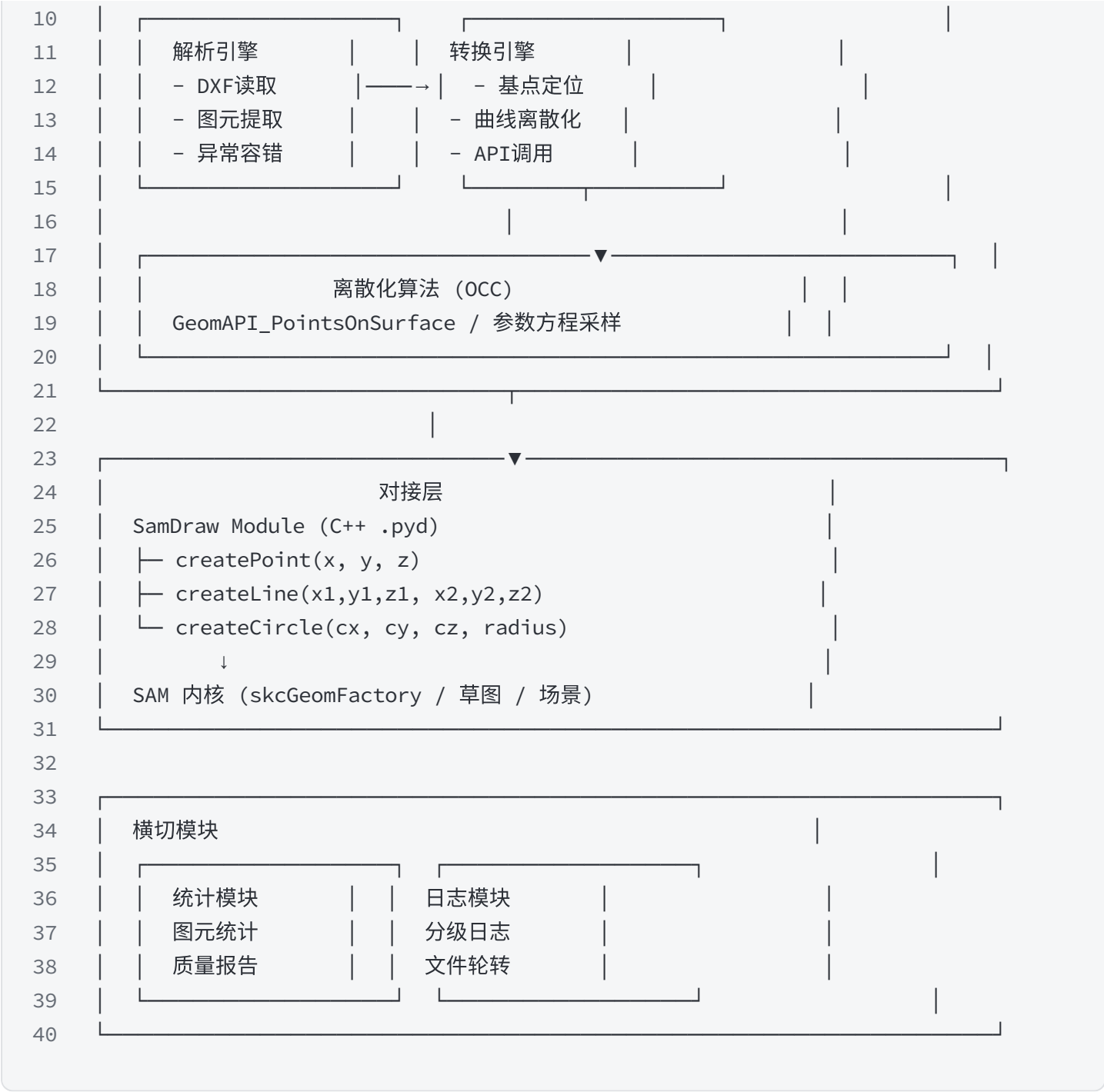
2.1 架构设计原则

原则	说明
关注点分离	DXF 解析、曲线处理、SAM 对接三层独立，各自只关注本层职责
数据驱动	层与层之间通过标准数据格式通信，不共享内部状态
容错设计	每层对输入数据做合法性校验，异常时降级而非崩溃
渐进增强	核心管线 MVP 可用后，逐步增加复杂图元和高级功能

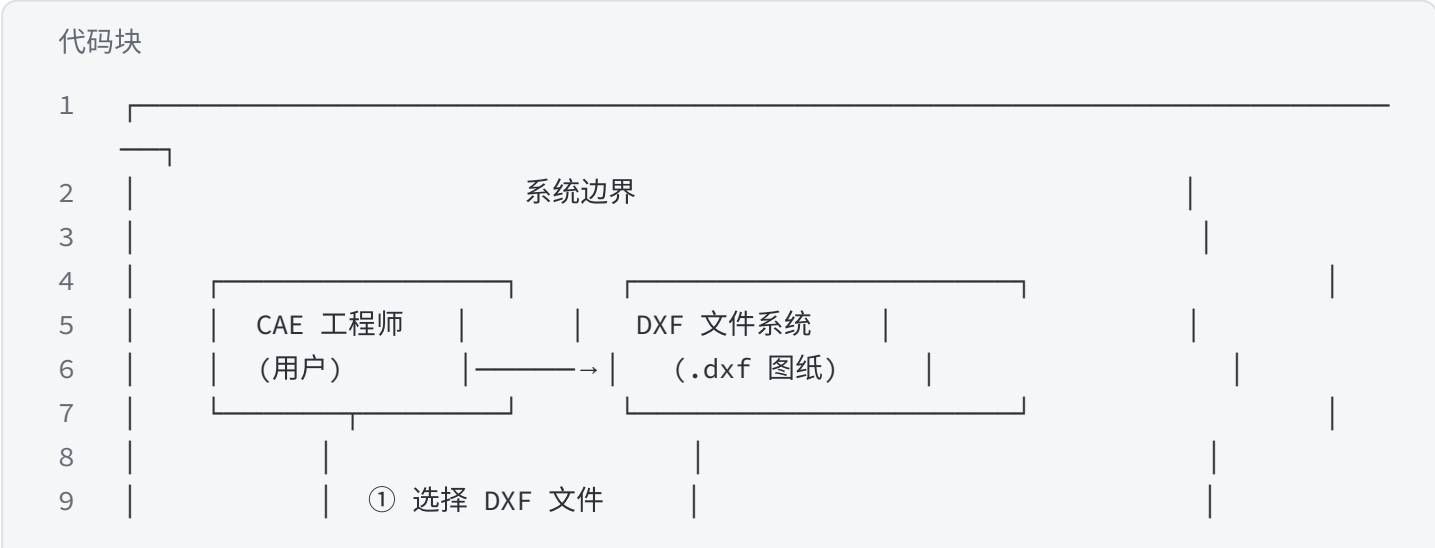
2.2 系统分层

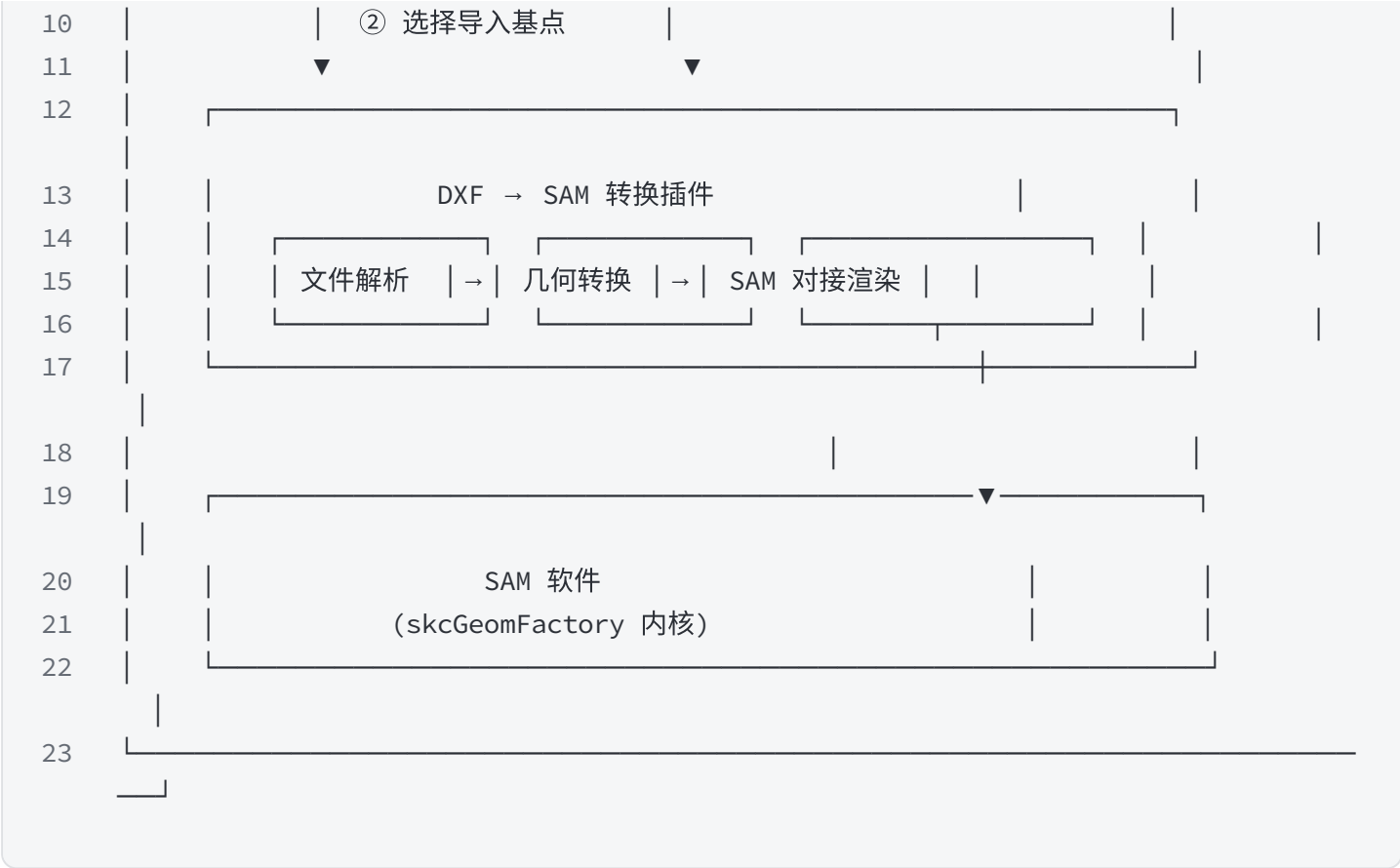
系统按功能划分为 3 层 + 3 个横切模块：



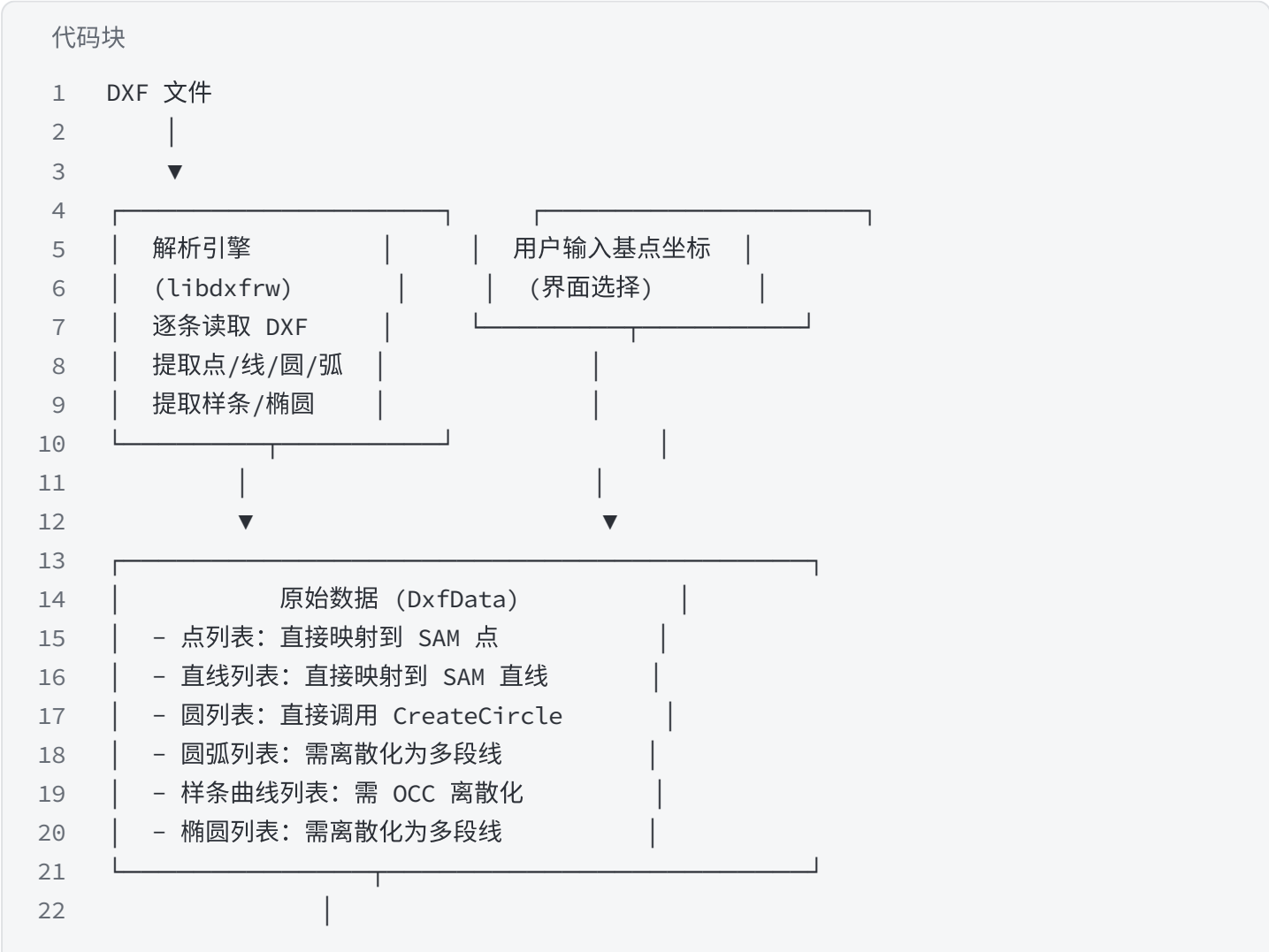


2.3 系统上下文图





2.4 数据流图





2.5 技术选型

层次	选型	选型理由
开发语言	C++17	SAM SDK 仅提供 C++ 接口；性能要求高
DXF 解析	libdxfw	纯 C++ 无外部依赖，支持 DXF R12~R2018，项目已集成
曲线离散化	OpenCASCADE (OCC)	专业的几何建模内核，提供成熟的样条曲线离散化接口
构建工具	CMake ≥ 3.5	跨平台，与 SAM 官方开发手册一致
GUI 框架	Qt 5.12.6	SAM 本身基于 Qt，保证 ABI 兼容
单元测试	Google Test	社区主流，适合算法验证
日志	自研 Logger 或 spdlog	轻量，支持分级和轮转

3. 模块设计

3.1 文件解析模块

职责范围： 读取 DXF 文件并提取几何图元信息。

处理流程：

代码块

```
1  用户传入 DXF 文件路径
2      ↓
3  Step 1: 文件前置校验
4      - 检查文件是否存在
5      - 检查文件是否可读
6      - 检查扩展名是否为 .dxf
7      - 检查是否为二进制格式（不支持则提示用户）
8      ↓ 失败 → 返回错误码 + 日志
9      ↓ 成功
10 Step 2: 注册 libdxfrw 回调
11     - addLine      → 收集直线
12     - addCircle    → 收集圆
13     - addArc       → 收集圆弧
14     - addSpline    → 收集样条曲线
15     - addLWPolyline → 收集多段线
16     - addEllipse   → 收集椭圆
17     ↓
18 Step 3: 驱动解析
19     - 逐条读取 DXF 记录
20     - 遇到异常数据 → 容错处理
21     ↓
22 Step 4: 输出 DxfData
```

异常容错策略：

异常类型	处理方式	用户可见反馈
文件不存在	中断解析，返回错误码	弹窗提示"文件不存在"
文件权限不足	中断解析，返回错误码	弹窗提示"权限不足"
文件被占用	中断解析，返回错误码	弹窗提示"文件被占用"
DXF 格式损坏	跳过损坏记录，继续解析	日志记录跳过的位置和原因

零长度直线	跳过该图元，不加入数据	日志记录被跳过的图元 ID
半径为零的圆弧	跳过该图元，不加入数据	日志记录被跳过的图元 ID
字段缺失	用默认值替代该字段	日志记录警告
二进制格式 DXF	中断解析，提示不支持	弹窗提示"不支持二进制格式 DXF"

3.2 曲线离散化模块（基于 OCC）

职责范围： 将圆弧、样条曲线、椭圆等曲线图元离散化为 SAM 能处理的直线段集合。

本模块基于 OpenCASCADE（OCC）数学库实现曲线离散化。OCC 提供了成熟的几何处理接口，能够保证离散化精度和稳定性，避免了自研算法的维护成本。

圆与圆弧的处理策略

- 完整圆（360° 闭合）：** 不进行离散化，直接调用 SAM 的 CreateCircle 接口。这是最优方案，因为 SAM 原生支持圆，且能保证精度和性能。
- 非完整圆弧：** SAM 的 CreateCircle 不支持画圆弧，因此需要将圆弧离散化为多段线。基于 OCC 的 `GeomAPI_PointsOnSurface` 或参数方程采样，将圆弧按角度步长（默认 5°）采样为点列，再通过 CreateLine 逐段连接。

样条曲线的离散化

样条曲线（Spline/NURBS）是本项目的难点。采用 OpenCASCADE 库提供的离散化接口进行处理：

- OCC 的 `GeomAPI_PointsOnCurve` 或 `BRepBuilderAPI_MakeEdge` 结合 `BRepTools` 可以直接将样条曲线离散化为指定精度的多段线
- OCC 内部采用自适应的离散化算法，能够自动在曲率大的区域加密采样点，在平直区域减少采样点
- 离散化精度通过参数控制（默认 0.1mm）

使用 OCC 的优势：

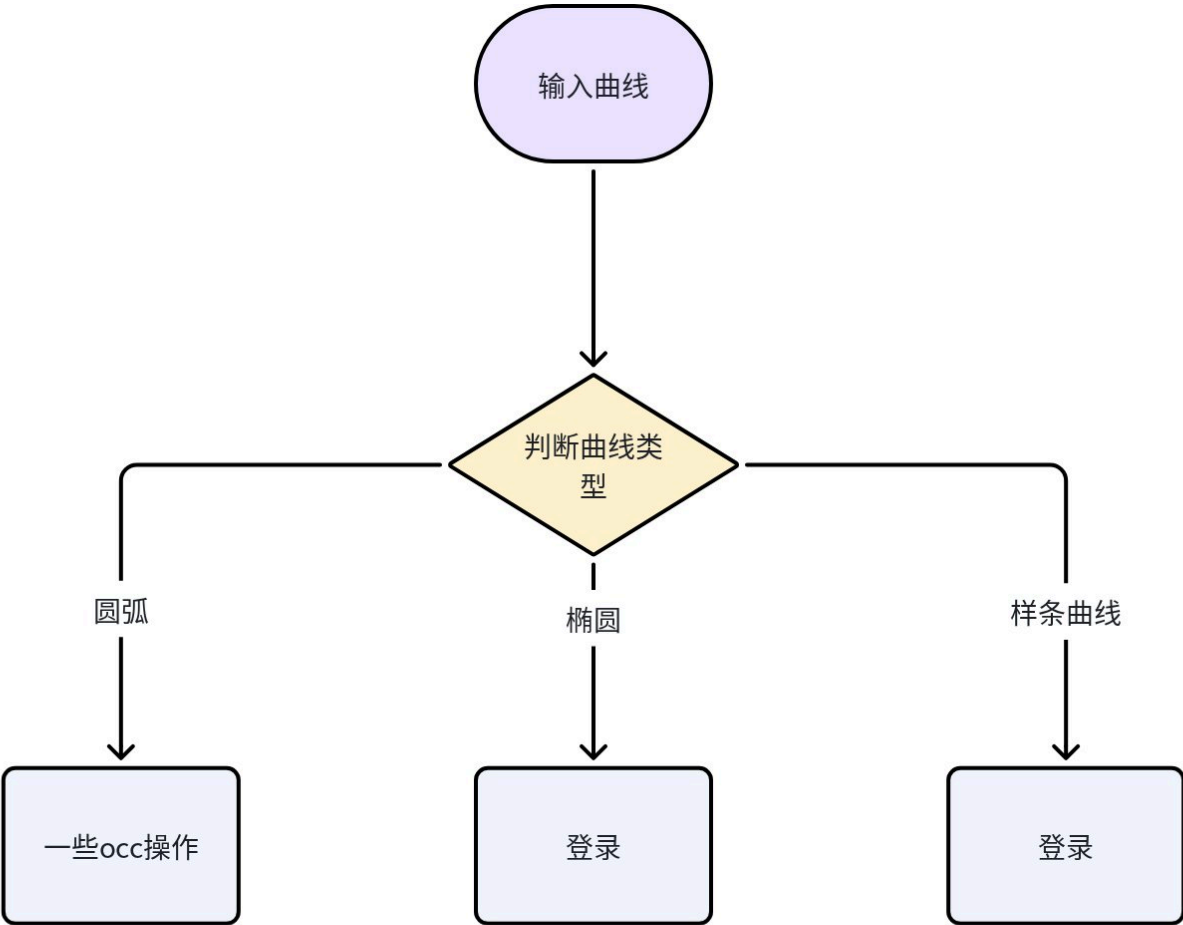
- 成熟可靠：** OCC 是工业级几何内核，离散化算法经过大量验证
- 精度可控：** 直接控制离散化偏差上限
- 省去自研成本：** 无需实现 Douglas-Peucker 等算法，避免重复造轮子

参数	默认值	说明
离散化容差	0.1 mm	越小越精确，但点数越多
角度步长（圆弧）	5°	控制圆弧离散化密度

椭圆/椭圆弧的离散化

椭圆基于 OCC 的 `Geom_Ellipse` 类，通过参数方程采样实现离散化。OCC 提供了 `Adaptor3D_Curve` 配合 `GeomAdaptor_Curve` 实现均匀或自适应采样。

曲线拟合流程图



代码块

```
1  |-----|
2  |           曲线离散化流程           |
3  |-----|
4  |  输入：曲线图元（圆弧/样条/椭圆）+ 容差参数  |
5  |-----|
6  |           |           |           |
7  |  [ 判断曲线类型  ] |           |
8  |           |           |           |
9  |           |           |           |
10 |-----|-----|-----|
11 |           |           |           |
12 |  [ 圆弧 ] | [ 样条曲线 ] | [ 椭圆 ] |
13 |           |           |           |
14 |-----|-----|-----|
15 |           |           |           |
16 |           |           |           |
```


船舶 CAD 图纸的坐标原点可能远离模型本身（例如大地坐标系下的偏移），而 SAM 希望在模型原点附近操作。因此提供用户手动选择导入基点的功能：

- **输入方式：**在 GUI 界面中输入基点的 X、Y、Z 坐标值，或在 SAM 场景中拾取一个点作为基点
- **效果：**CAD 图纸的原点 (0,0,0) 将被放置到用户指定的基点位置
- **默认值：**若用户未指定，默认以 (0,0,0) 为基点

这样做相比自动归一化的优势：

- 用户对导入位置有完全的控制权
- 避免自动偏移导致的意外结果
- 适合分块图纸的精确拼接场景

3.4 统计模块

职责范围：统计转换过程中的各类数据，生成可读报告。

报告内容分为以下维度：

- **按图元类型统计：**各类图元的原始数量、离散化后的数量变化
- **异常统计：**被跳过的异常图元数量、类型和原因
- **质量指标：**离散化的最大偏差

输出格式支持界面直接展示和日志文件保存两种方式。

3.5 日志模块

日志模块提供四个级别输出：

级别	颜色	使用场景
DEBUG	灰色	开发调试信息，如"正在离散化样条曲线"
INFO	绿色	正常流程信息，如"解析完成，共 209 个图元"
WARNING	黄色	可恢复异常，如"跳过零长度直线 (id=42)"
ERROR	红色	不可恢复异常，如"文件不存在，errno=2"

功能设计要求：

- 控制台和日志文件双路输出

- 日志文件按大小轮转（每 10 MB 分割，保留最近 5 个）
- 崩溃时自动 flush，确保最后一条日志不丢失
- 显示每一个元素导入是否成功，弧线是否转换成多段线，转换后多了哪些点与线

3.6 SAM 对接模块

职责范围： 对接 SamDraw 模块（C++ .pyd），完成最终的几何绘制。

该模块是对 SamDraw 模块的上层封装。SamDraw 模块已提供三个核心接口：

代码块

```
1  createPoint(x, y, z)          → 返回点 ID
2  createLine(x1,y1,z1, ...)     → 返回线 ID
3  createCircle(cx,cy,cz, r)     → 返回圆 ID
```

对接模块在此基础上增加：

- **草图生命周期管理：** 每次导入使用新的独立草图，名称基于时间戳，不干扰已有内容
- **批量提交：** 所有图元绘制完成后统一刷新场景

4. 数据设计

4.1 核心数据模型

系统定义两类数据容器，分别服务于解析阶段和转换阶段。

解析数据（DxfData）—— 解析引擎的输出：

代码块

```
1  struct DxfData {
2      vector<Point>    points;        // 点列表
3      vector<Line>     lines;         // 直线列表
4      vector<Circle>   circles;       // 圆列表（含完整圆）
5      vector<Arc>      arcs;          // 圆弧列表（需离散化）
6      vector<Spline>   splines;       // 样条曲线列表（需 OCC 离散化）
7      vector<Polyline> polylines;     // 多段线列表
8      vector<Ellipse>  ellipses;      // 椭圆列表（需离散化）
9  };
```

就绪数据（SamData）—— 转换引擎的输出（可直接对接 SAM）：

代码块

```
1 struct SamData {
2     vector<Point>    points;    // 点列表（与 DxfData 中的点一一对应）
3     vector<Line>     lines;     // 直线列表（含原始直线 + 离散化曲线产生的线段）
4     vector<Circle>   circles;   // 圆列表（仅保留完整的圆）
5 };
```

数据流转关系：

```
代码块
1 DXF 文件 → DxfData（与解析库耦合，保留原始几何信息）
2   ↓ 基点平移 + OCC 离散化
3 SamData（仅含点/线/圆，与 SAM 对接层耦合）
4   ↓ SAM API 调用
5 SAM 场景中的模型
```

4.2 容器选型

图元数据的存储以逐一遍历为主要操作场景，因此优先选用 `std::vector` 顺序容器：

- 遍历性能好，内存连续，CPU 缓存友好
- 图元数量通常在 10^5 以内，顺序查找足够
- 需要按 ID 快速查找时，额外建立索引表

4.3 模块间接口

接口标识	调用关系	传入数据	传出数据	异常情形
I-Parse	主控 → 解析引擎	文件路径	DxfData / 错误码	文件不存在/权限不足返回错误码
I-Discrete	转换引擎 → 离散化模块	曲线数据 + 容差	折线顶点	点数不足返回原数据
I-Convert	主控 → 转换引擎	DxfData + 基点坐标	状态码	空数据不操作
I-Report	主控 → 统计模块	DxfData + SamData	报告文本	无数据返回空报告
I-Log	全局使用	格式化字符串	写入文件/控制台	写失败不中断主流程

4.4 与 SAM 的外部接口

接口	输入	输出	异常处理
读取 DXF 文件（文件 I/O）	文件路径	文件流 / 错误码	不存在返回 -1，权限不足返回 -2
createPoint	x, y, z	点 ID	参数越界返回 -1
createLine	起点 + 终点坐标	线 ID	退化线返回 -1
createCircle	圆心 + 半径	圆 ID	半径 ≤ 0 返回 -1

5. 关键设计决策

5.1 圆与圆弧的分合策略

决策项	决策内容	理由
完整的圆	直接调用 SAM 的 CreateCircle	SAM 原生支持，精度和性能最好
非完整圆弧	离散化为多段线后逐段画线	SAM 不提供画圆弧接口，只能折线逼近

5.2 离散化方案的选择：OCC vs 自研算法

对比项	OpenCASCADE（选用）	自研 Douglas-Peucker（备选）
实现成本	直接调用 API	需要自研实现、测试、维护
可靠性	工业级验证，经过大量用户检验	需自行验证边界情况
精度控制	OCC 内部自适应，精度可靠	需自行实现精度控制逻辑
算法复杂度	封装在 OCC 内部	$O(n \log n)$
适用场景	各类曲线（样条、圆弧、椭圆）	主要适用于样条曲线

结论： 选用 OpenCASCADE 作为离散化引擎，圆弧采用 OCC 参数方程采样，样条曲线和椭圆采用 OCC 自适应离散化。各取所长且避免重复造轮子。

5.3 坐标处理方案：用户选点 vs 自动归一化

--	--	--

对比项	用户选点导入基点（选用）	自动坐标归一化（备选）
控制权	用户完全控制导入位置	系统自动决策
适用场景	分块图纸拼接、精确对齐	单张图纸快速导入
实现复杂度	仅需平移变换	需计算包围盒 + 单位检测 + 平移
风险	用户需理解基点概念	自动缩放可能不符合预期

结论： 采用用户选点导入基点的方案。CAD 图纸的零点位置由用户在界面上指定，系统将所有图元坐标以该基点为原点进行平移。若用户不指定，默认使用 (0,0,0)。

5.4 插件化集成方案

决策项	决策内容	理由
集成方式	以插件形式嵌入 SAM 软件	需求明确要求插件形式
GUI 方案	基于 Qt 的图形界面	SAM 本身基于 Qt，ABI 兼容
版本假设	假设 DXF 文件为 2018 版本	其他版本不能保证正常使用
坐标系假设	假设用户使用世界坐标系，单位为毫米	自定义坐标系需用户先转换

6. 测试策略

6.1 测试层次

单元测试层： 针对离散化算法、数据转换等核心功能，使用 Google Test 框架。重点覆盖边界条件：零长度直线、零半径圆弧、极端坐标值、空数据。

集成测试层： 准备一组标准测试 DXF 文件，按复杂度递增排列：

测试文件	内容特征	验证重点
test_simple.dxf	仅含直线和圆	核心管线连通性
test_curves.dxf	圆弧 + 样条曲线	OCC 离散化精度
test_ship.dxf	真实船舶图纸片段	整体可靠性
test_malformed.dxf	含异常数据	容错能力

test_empty.dxf	空文件	稳定性
----------------	-----	-----

端到端测试： 在 SAM 中加载转换结果，目视验证几何完整性，确认没有线条缺失、坐标偏移、ID 冲突。

6.2 验收标准

指标	目标值
离散化最大偏差	≤ 设定容差的 1.5 倍
100MB 文件解析时间	≤ 30 秒
异常图元跳过程	< 总图元的 1%
程序崩溃率	0%（不因任何输入崩溃）

6.3 测试要点

- 基本图元测试：** 只有点、只有直线、只有圆（圆弧）、只有样条曲线，以及多种图元混合。重点在于是否成功导入，导入后是否有线条与点的错乱情况。
- 大文件测试：** 100MB 以上的 DXF 文件。重点在于导入时间、是否卡死。考虑异步导入方案，期间若进行其他操作不应导致数据异常或软件崩溃。加载框显示的内容需正确。
- 异常文件测试：** 文件不存在、权限不足、文件被占用、文件内容损坏、空文件。重点在于日志的正确报错信息，弹窗的错误描述。
- 连续导入测试：** 连续导入多个 DXF 文件。检查是否会卡死崩溃，是否有两个不同点共用一个 ID 的情况。
- 与 SAM 交互测试：** 导入 DXF 后在 SAM 中进行操作（创建点/线/圆、移动、删除），以及先在 SAM 中操作再导入 DXF。检查是否崩溃，是否产生 ID 冲突。
- 可视化界面测试：** GUI 各控件的功能完整性。

7. 迭代开发计划

按照 MVP 思路，分三阶段迭代：

第一迭代：核心管线打通（MVP）

目标： 实现从 DXF 到 SAM 的基本通道，能处理最简单的图纸。

包含功能点：

- 直线、圆的解析与绘制
- 基本统计输出（图元数量）
- 图形界面（文件选择、导入按钮）
- 日志系统
- 单元测试 + 集成测试

交付物： SAM 插件，可导入仅含直线和圆的 DXF，在 SAM 中正确画出对应图形。

第二迭代：复杂图纸支持

目标： 支持真实船舶图纸，含曲线和图块（需用户手动展开）。

包含功能点：

- 圆弧离散化（OCC）
- 样条曲线离散化（OCC）
- 椭圆/椭圆弧支持
- 基点选择功能
- 倒角处理
- 多维度统计报告
- 大文件异步处理

交付物： 可完整转换真实船舶图纸，曲线均正确还原。

第三迭代：产品化完善

目标： 提供完整的产品化体验。

包含功能点：

- 批量文件处理
- 配置文件系统（持久化参数）
- 安装包制作
- 异常场景全面覆盖
- 性能优化

交付物： 带 GUI 的 SAM 插件，含安装程序。

8. 风险与应对

--	--	--	--

风险	发生概率	影响程度	应对措施
libdxfrw 无法解析特定版本 DXF	中	高	在解析层做抽象隔离，可替换为其他解析库
OCC 离散化精度不满足要求	低	中	提供容差参数由用户调节
SAM API 版本变更导致不兼容	低	高	与业务层严格解耦，优先使用通用中间数据
超大图纸（>500MB）内存溢出	中	中	流式解析 + 分块处理 + 内存监控
中文路径/文件名乱码	中	中	统一内部编码为 UTF-8，覆盖常见编码检测